

PyTorch Profiling

Introductory guide to profiling PyTorch code and optimising performance.

Rosie Wood · Iain Stenson · Tomas Lazauskas · The Alan Turing Institute

Introduction

Why Profile?

- **GPU time is expensive** — wasted cycles mean wasted money, energy, and CO₂
- **Bottlenecks are non-obvious** — the slow part is rarely where you expect it
- **Small fixes, big gains** — correct data loading or a single flag can yield big speedups
- **Scale amplifies inefficiency** — a 10% overhead at 1 GPU becomes critical across 100

"If you can't measure it, you can't improve it."

Session Goal

#	Topic
0	PyTorch distributed recap
1	Single-GPU baseline
2	GPU utilisation with nvidia-smi
3	Model FLOP Utilisation (MFU)
4	torch.profiler + HTA + Perfetto
5	Nsight Systems (<i>bonus</i>)

Exercise 5 is a bonus for those already familiar with the tools. Don't worry if you don't finish everything.

github.com/tomaslaz/tutorial-pytorch-prof

Exercise Structure

Each exercise is self-contained in its own directory:

```
1  N_topic/
2  |— README.md      # instructions and questions to consider
3  |— train.py        # training script to modify
4  |— launch.py       # multi-GPU launch script
5  |— sbatch.sh       # job submission script
6  |— solution/       # complete solution with answer.md
```

Start by reading the `README.md`, modify `train.py` and/ or `sbatch.sh`, and submit with `sbatch.sh`. Check `solution/` if you get stuck and for discussion of the results in `answer.md`.

github.com/tomaslaz/tutorial-pytorch-prof

Key Tools

Tool	Purpose	Exercise
<code>`time`</code>	Simple wall-clock timing	1
<code>`nvidia-smi`</code>	GPU utilisation, memory, and power at a glance	2
<code>`FlopCounterMode`</code>	Counts FLOPs for PyTorch operations	3
<code>`torch.profiler`</code>	Timeline of CPU/GPU ops, memory, and stack traces	4
Nsight Systems	System-wide GPU/CPU timeline	5
Nsight Compute	Deep kernel-level metrics; roofline, memory throughput	— (<i>not covered</i>)

github.com/tomaslaz/tutorial-pytorch-prof

Key Concepts

FLOPs — count of floating point operations

FLOP/s — hardware throughput; GH200 = 989 TFLOP/s (bf16)

MFU — actual FLOP/s $\div$ peak FLOP/s

Throughput — samples or tokens per second

Step time — wall-clock time per iteration

Arithmetic intensity — FLOPs per byte accessed

Compute-bound vs memory-bound — is the bottleneck arithmetic or data movement?

Memory bandwidth — data transfer rate between HBM and compute

Peak memory — VRAM consumed; determines max batch size

GPU utilisation — fraction of SMs active; high % $\neq$ efficient use

Data loading time — CPU/IO bottleneck that starves the GPU

Tensor Cores — activated by bf16/fp16; far higher throughput than FP32 cores

Communication/compute overlap — does NCCL all-reduce hide behind the backward pass?

github.com/tomaslaz/tutorial-pytorch-prof

Our Running Example

BERT fine-tuning with DDP on Isambard-AI — from the previous session

```
1  def benchmark():
2      ...
3      model = BertForSequenceClassification.from_pretrained(model_name).to(DEVICE)
4      model = DDP(model, device_ids=[local_rank])
5
6      dist.barrier()
7      start_time = time.time()
8
9      for _ in range(TRAINING_STEPS):
10         optimizer.zero_grad()
11         outputs = model(**inputs, labels=labels) # forward pass
12         loss = outputs.loss
13         loss.backward()                         # backward pass
14         optimizer.step()
15
16     dist.barrier()
17     end_time = time.time()
```

github.com/tomaslaz/tutorial-pytorch-prof

Our Running Example

Running on **2 nodes, 8 GPUs** (Isambard-AI):

```
1 Time taken for 2 forward and backward pass(es) with BATCH_SIZE=32 on 8 workers: 0.795 seconds
```

We measured total step time — but:

- How much is **compute** vs **communication**?
- Are the GPUs actually busy?
- Where is the time going?

→ Let's profile it.

Start working on the tutorial:

github.com/tomaslaz/tutorial-pytorch-prof

Hands-on Time

github.com/tomaslaz/tutorial-pytorch-prof